

3. Method

3.1. Device simulation and dataset construction

3.1.1. The constant-capacitance model

Each simulated device consists of a double quantum dot (DQD) capacitively coupled to a single charge sensor. It is modelled with the QArray simulator within the constant-capacitance approximation, and the equivalent circuit is shown in **Fig. 1(a)**. In this approximation the device is treated as a network of fixed capacitors. For a given gate-voltage vector V_g , the ground-state charge configuration $n = (n_1, n_2)$ is the integer occupation that minimises the electrostatic energy of the network. The transition lines of the charge stability diagram are the lines in gate-voltage space at which two charge configurations are degenerate; crossing such a line changes the number of carriers on the dots. Across each such line the ground-state occupation changes discretely from one integer configuration to another.

Each of the three conducting islands (two dots and the sensor) couples to each of the three gates, giving nine capacitive branches, and the islands couple to one another through c_{d1d2} , c_{s1d1} and c_{s1d2} , giving three more, so twelve branches are drawn in total in **Fig. 1(a)**. The two remaining parameters, c_{d1d1} and c_{d2d2} , are self-capacitances and are not shown in the drawing, for simplicity, so a device is specified by 14 capacitances in all.

The same capacitances are collected into the four matrices of **Fig. 1(b)**. The dot–dot matrix $C_{dd} \in \mathbb{R}^{2 \times 2}$ carries the dot self-capacitances c_{d1d1} and c_{d2d2} on its diagonal and the interdot mutual capacitance c_{d1d2} off it; the dot–gate matrix $C_{dg} \in \mathbb{R}^{2 \times 3}$ has rows indexed by dot and columns by gate, with entries c_{digi} ($i = 1, 2; j = 1, 2, 3$); the sensor–dot matrix $C_{sd} \in \mathbb{R}^{1 \times 2}$ has entries (c_{s1d1}, c_{s1d2}) ; and the sensor–gate matrix $C_{sg} \in \mathbb{R}^{1 \times 3}$ has entries $(c_{s1g1}, c_{s1g2}, c_{s1g3})$. Because mutual capacitance is reciprocal, $c_{d1d2} = c_{d2d1}$ and C_{dd} is symmetric, so the 15 entries displayed hold the same 14 independent parameters: three in C_{dd} , six in C_{dg} , two in C_{sd} and three in C_{sg} .

Each device realisation corresponds to a single draw of this 14-dimensional parameter vector from uniform distributions over the ranges listed in **Table I**.

Each of the 14 capacitance entries is drawn independently and uniformly from the intervals in **Table I**, so that every device corresponds to a point in a 14-dimensional parameter space.

The sensor response is computed with the noise model set to NoNoise and nothing is added afterwards, so the maps are noise-free. The Coulomb peak width is 0.01 and the electron temperature $1e-05$ in simulation units.

Each device is rendered over a 2×2 mV window at 100×100 pixels. The window position is shifted at random for each device, by up to 35 % of its width along each axis, so that the transition lines do not appear at fixed positions in the frame.

Figure 2 shows the charge-sensor signal (a) and the transition-line map (b) that the simulator returns for one device.

3.1.2. Train/test split

A total of 550 devices are generated by sampling the capacitance intervals of **Table I**. The dataset is split by device: 500 devices are used for training and 50 for testing. Before training, 15 % of the training devices (75 of 500) are set aside as a validation set, and the network weights are fitted on the remaining 425 devices. The validation set is used to select the best-performing epoch and the binarisation threshold (Section 3.4). The test devices are not used for any training or model-selection decision. **Figure 3** illustrates this partitioning.

3.2. Ray-based measurement and network input

3.2.1. The ray fan

A measurement budget is a pair $(n_{\text{rays}}, n_{\text{points}})$. The rays emanate from the maximum-voltage corner $(V_1^{\text{max}}, V_2^{\text{max}})$ of the window, the k -th ray propagating along the inward unit vector $\mathbf{d}_k = (-\cos \theta_k, -\sin \theta_k)$ with

$$\theta_k = 90^\circ k / (n_{\text{rays}} + 1), \quad k = 1, \dots, n_{\text{rays}}.$$

The angles are equispaced on the open interval $(0^\circ, 90^\circ)$; the endpoints are excluded because an axis-aligned ray runs along the window boundary and intersects few transition lines. For $n_{\text{rays}} = 8$ this gives $\theta \in \{10^\circ, 20^\circ, 30^\circ, \dots, 80^\circ\}$ (**Figure 2(c)**).

Each ray corresponds to a single one-dimensional voltage sweep. We chose this ray-based pattern primarily for its simplicity. By distributing the rays across the voltage window, we aimed to ensure that each ray intersects at least one charge transition line, so that every sweep carries information about the underlying stability diagram. We restrict all ray angles to the open interval $(0^\circ, 90^\circ)$ so that the rays spread across the interior of the window rather than running along its edges. For simplicity, we space the rays at equal angles and use the same number of points on every ray. We also use the same set of rays for every device, rather than adapting it to what we observe during the measurement. We did not optimise any of these choices. To examine how the measurement budget affects performance, we evaluate 4, 5, 6, 7, and 8 rays, each sampled at 40, 50, or 60 points per ray, giving 15 configurations in total. For each configuration, we use 550 samples, of which 500 serve for training and 50 for testing.

The measurement budget is defined as the total number of single-point measurements.

Each measured voltage pair is assigned to the pixel whose centre lies closest to it. Because all rays share a common origin, the first point of every ray falls in the same pixel, and adjacent rays may also be assigned to a common pixel in the following step. The number of distinct pixels sampled is therefore slightly smaller than the number of measurements. For the configuration of eight rays of 60 points, the 480 measurements would cover $480/(100 \times 100) = 4.80\%$ of the grid if every pixel were distinct, but they occupy 466 distinct pixels, or 4.66 %. Since a repeated measurement of the same pixel provides no additional information to the network, coverage is quantified throughout this work as the fraction of distinct pixels sampled.

3.2.2. Input representation

The network receives two input channels, each with the dimensions of the full stability diagram. The first contains the sensor signal, min–max normalised to $[0, 1]$ for each device, at every pixel traversed by a ray, and zero at all other pixels. The second is a binary mask that is one at every sampled pixel and zero otherwise. A single channel would not suffice. A convolutional network requires a value at every pixel, so the unsampled pixels must be assigned a placeholder, and after normalisation zero is also the value taken by the lowest measured signal in each diagram. In the signal channel alone, a zero would therefore be ambiguous: it could be a sampled pixel with low sensor signal or a pixel that was never measured. The network could not separate the absence of a measurement from a measurement indicating the absence of a transition. The mask removes this ambiguity by specifying where measured information exists and where the prediction must be inferred from neighbouring rays. Because both channels always span the full diagram, the input dimensions and the network architecture are identical for every measurement budget, and only the density of sampled pixels changes.

Figure 2(c) and (d) show the two channels for one held-out device.

An important property of this encoding is that the input dimensions are independent of the measurement budget. Changing n_rays or n_points alters only the fraction of pixels populated in the two channels, not the size of the channels themselves. The same network architecture, with the same number of trainable parameters, can therefore be used for every budget. A separate model is trained for each budget, but because the architecture and training procedure are identical, differences in performance across budgets can be attributed to the measurement rather than to differences in model capacity, within the run-to-run variation of training.

3.3. Network architecture and training

3.3.1. Architecture

Figure 4 shows the architecture of the fully convolutional U-Net. The input is a two-channel 100×100 tensor (sensor signal and sampling mask). Each encoder stage (32, 64 and 128 channels) consists of two 3×3 convolutions, each followed by batch normalisation and a GELU activation, and is followed by 2×2 max pooling ($100 \rightarrow 50 \rightarrow 25 \rightarrow 12$). The bottleneck has 256 channels. In the decoder, feature maps are upsampled by nearest-neighbour interpolation to the size of the corresponding encoder output and concatenated with it through skip connections. A final 1×1 convolution with a sigmoid produces a per-pixel transition-line probability map of size 100×100 . Extracting lines from this map is a separate decision step, described in Sec. 3.4.

3.3.2. Loss function

Transition-line pixels make up only a small fraction of each diagram. A network that predicts no transition anywhere would therefore still be correct for most pixels while finding no lines at all. Two terms in the training loss prevent this. First, the per-pixel binary cross-entropy is weighted so that a missed transition-line pixel costs more than an error on a pixel without a line. Second, a Dice term compares the predicted and true line maps as a whole and penalises predictions that place lines where there are none.

3.3.3. Model selection and evaluation metric

After each training epoch, the network was evaluated on the validation set, and the weights from the epoch with the highest validation score were retained as the final model rather than those from the last epoch. Performance is measured with the F1 score, which combines precision (the fraction of predicted line pixels that are correct) and recall (the fraction of true line pixels that are recovered) into a single number between 0 and 1. Because the transition lines are only one pixel wide, a predicted pixel is counted as correct if it lies within a small distance, the tolerance τ , of a true line pixel. Model selection uses a tolerance of one pixel (F1@1). The motivation for this tolerance, and its effect on the reported scores, are discussed in Sec. 3.4.

For four of the fifteen budgets, 5×60 , 6×50 , 7×60 , and 8×60 , training did not leave its initial plateau: the best validation F1@1 remained between 0.42 and 0.44 over all 50 epochs. These budgets were retrained from a different random initialisation of the weights, with the training data, architecture and training procedure unchanged.

3.4. From probability map to transition lines

The network assigns each pixel a probability that a charge-transition line passes through it. To obtain the transition lines themselves, this probability map is binarised by applying a threshold. Pixels whose probability exceeds the threshold are classified as lying on a

transition line, and all others as lying off it. The reconstruction of the transition lines thereby becomes a binary classification problem at the pixel level, and its performance can be quantified with the standard metrics of binary classification. A pixel predicted to lie on a transition line that does lie on one is a true positive (TP). A pixel predicted to lie on a transition line where none exists is a false positive (FP). A pixel on a true transition line that the prediction misses is a false negative (FN). From these we define the precision $P = TP/(TP + FP)$, the fraction of predicted transition pixels that are correct; the recall $R = TP/(TP + FN)$, the fraction of true transition pixels that are recovered; and their harmonic mean, the F1 score, $F1 = 2PR/(P + R)$. These quantities are used throughout the remainder of this work.

The value of the threshold determines how the errors are distributed between the two kinds. If the threshold is set low, even pixels with a small predicted probability are classified as lying on a transition line. Few true transitions are missed, so the recall is high, but many pixels are wrongly marked as transitions, so the precision is low. If the threshold is set high, only pixels predicted with high confidence are retained. The marked pixels are then mostly correct, so the precision is high, but faint transitions are discarded, so the recall is low. A suitable threshold must therefore be chosen to balance these two effects.

3.4.1. Binarising the output: the threshold

To obtain transition lines from the probability map, each pixel must be classified as lying on a transition line or not. This is done by applying a threshold to the predicted probabilities. It is therefore a free parameter of the method and must be fixed without reference to the test data. We select it on a validation set of 75 devices held back from the training set. A range of candidate thresholds is evaluated on these devices, and the value giving the highest score is stored with the trained model. This procedure does not use the held-out test devices at any stage. The validation devices come from the training distribution and are disjoint from the test set, so the threshold is determined in the same way as any other hyperparameter of the network. The procedure also matches the intended application. On a real device the true transition lines are unknown, so the threshold must be fixed before the measurement using simulated data alone, which is exactly what is done here. The validation curve (**Fig. 5(b)**) is nearly flat over a broad range of thresholds and decreases only at very high values, so the result is insensitive to the exact choice. **Figures 5(a), (c) and (d)** show a representative probability map binarised at the selected threshold and at a deliberately excessive one.

3.4.2. Scoring the output: the tolerance

Transition lines are one pixel wide. Comparing them with the ground truth pixel by pixel is unduly strict: a perfectly recovered line displaced by a single pixel shares no pixel with

the truth and scores zero, although the reconstruction is physically correct. **Figure 6** shows this case, constructed and scored with the same metric code used throughout: strict F1 = 0.00, and F1 at $\tau = 1$ equal to 1.00.

We therefore report F1 at a tolerance τ : a predicted pixel counts as correct if a true line pixel lies within τ pixels of it, and a true pixel counts as found if a predicted one lies within τ . Distances are Euclidean. **Figure 7** shows what τ admits around a single pixel: $\tau = 0$ is the same pixel only, $\tau = 1$ adds the four side neighbours, and a diagonal neighbour is 1.41 pixels away so it counts only from $\tau = 2$.

Figure 8 shows the effect of the tolerance on a single prediction. It is a magnified region of one held-out device, with the ground-truth lines in black, the network output binarised at the selected threshold in red, and the pixels within τ of the ground truth in grey. The prediction and the ground truth are identical in every panel; only the tolerance band grows from $\tau = 0$ to 3, and the precision, recall and F1 given above each panel rise with it. The panel for $\tau = 1$, the tolerance used for model selection and for the headline results, is highlighted.

Figure 9 shows F1 against τ together with the precision and recall behind it.

Tables

Table I. Sampling intervals for the capacitance matrices of **Fig. 1(b)**, in QArray's dimensionless capacitance units. For each device, every capacitance entry is drawn independently from a uniform distribution over its interval. No sampled device is filtered or rejected. Training and test devices are drawn from the same intervals and are separated by device identity only.

Matrix	Entry	Interval
C_{dd}	d1d1	0.30 - 1.40
C_{dd}	d2d2	0.30 - 1.40
C_{dd}	d1d2	0.20 - 2.00
C_{dg}	d1g1	0.80 - 6.00
C_{dg}	d2g2	0.80 - 6.00
C_{dg}	d1g2	0.05 - 0.60
C_{dg}	d2g1	0.05 - 0.60
C_{dg}	d1g3	0.01 - 0.10
C_{dg}	d2g3	0.01 - 0.10
C_{sd}	s1d1	0.02 - 0.15
C_{sd}	s1d2	0.02 - 0.15
C_{sg}	s1g1	0.02 - 0.10
C_{sg}	s1g2	0.02 - 0.10
C_{sg}	s1g3	0.10 - 1.00

Figures

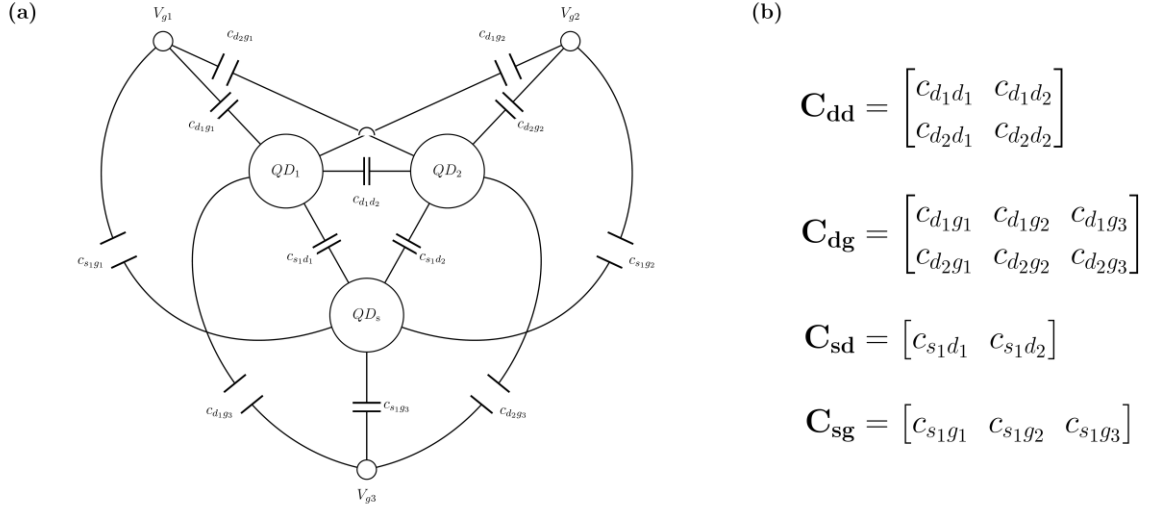


Fig. 1. The constant-capacitance model of the simulated device. (a) The capacitor network: two quantum dots QD1 and QD2, a sensor dot QDs, and three gates. (b) The four capacitance matrices that define a device. A device is one uniform draw of the 14 free parameters of **Table I**.

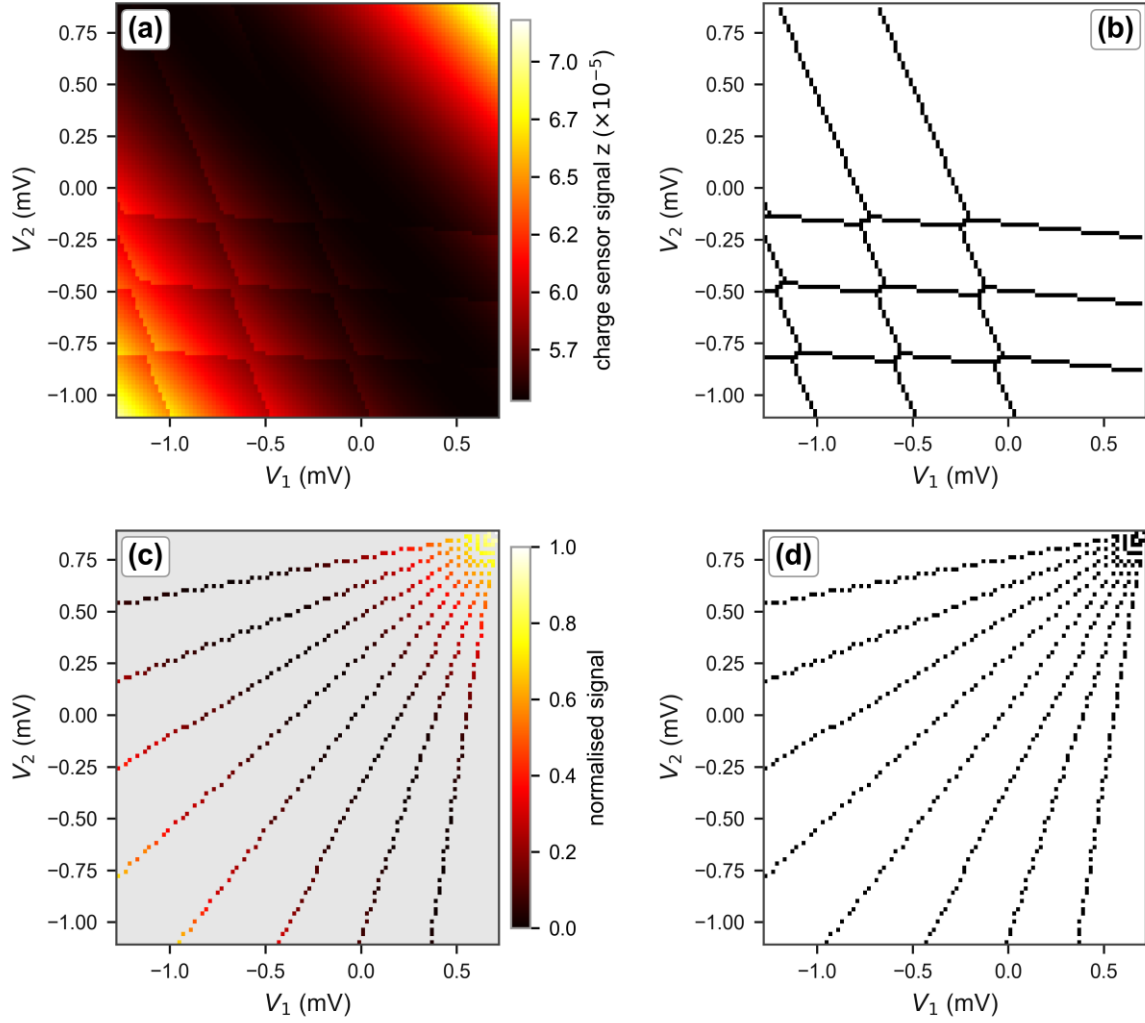


Fig. 2. One held-out device at the headline budget ($8 \text{ rays} \times 60 \text{ points}$). (a) The charge-sensor signal over the gate-voltage window. (b) The ground-truth transition lines computed by the simulator: the output the U-Net is trained to reproduce. (c) The normalised sensor signal at the pixels sampled by the rays, grey elsewhere: the first input channel. (d) The binary sampling mask: the second input channel.

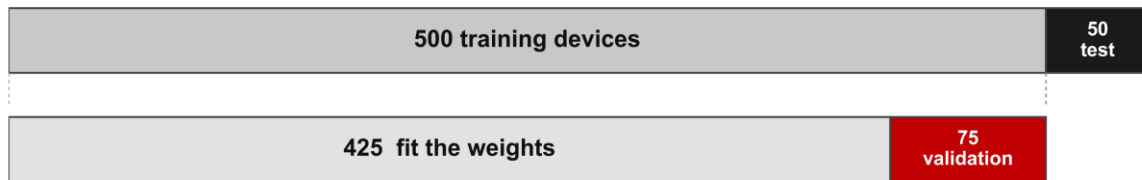


Fig. 3. Partitioning of the dataset. The 550 simulated devices are divided at the device level into a training set of 500 devices and an independent test set of 50 devices. Before training begins, 15% of the training devices are reserved for validation, so the model weights are fitted on the remaining 425 devices.

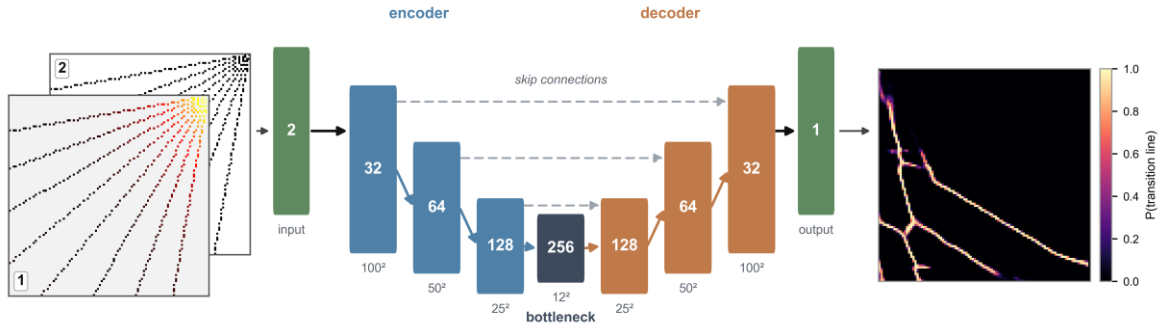


Fig. 4. Network input and output for one held-out device (8 rays $\times$ 60 points). Left: normalised sensor signal along the rays and the binary sampling mask. Centre: U-Net. Right: predicted 100×100 transition-line probability map.

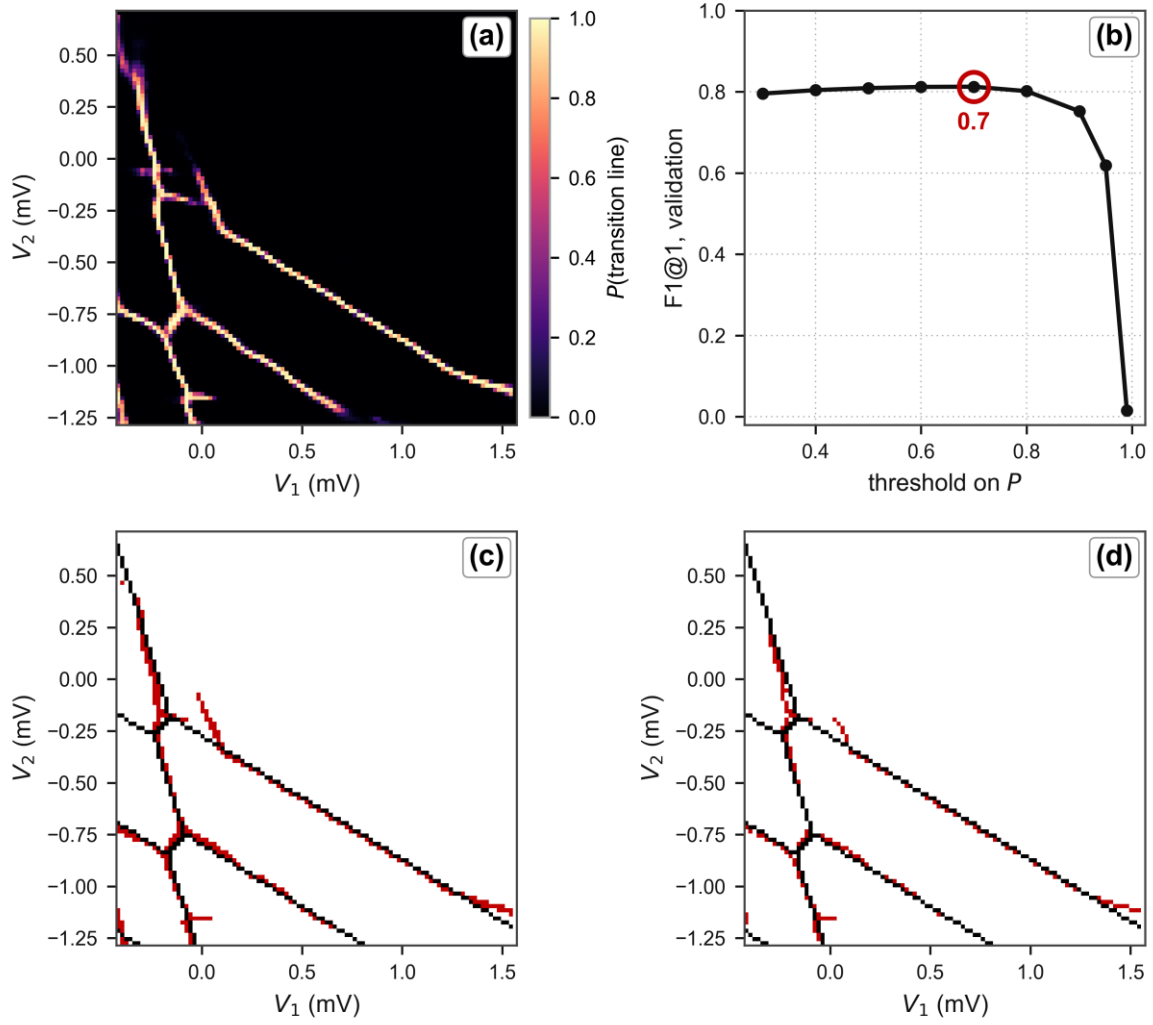


Fig. 5. Choosing and applying the threshold. (a) The U-Net output for one held-out device: the probability per pixel that a transition line passes through it. (b) Mean F1 at $\tau = 1$ over the 75 validation devices for every candidate threshold, with the stored value ringed. (c) The same map as (a) binarised at the selected threshold, $P > 0.6$, and (d) at a deliberately excessive one, $P > 0.9$, at which lines break up and are lost. In (c) and (d) the prediction is red and the ground truth black.

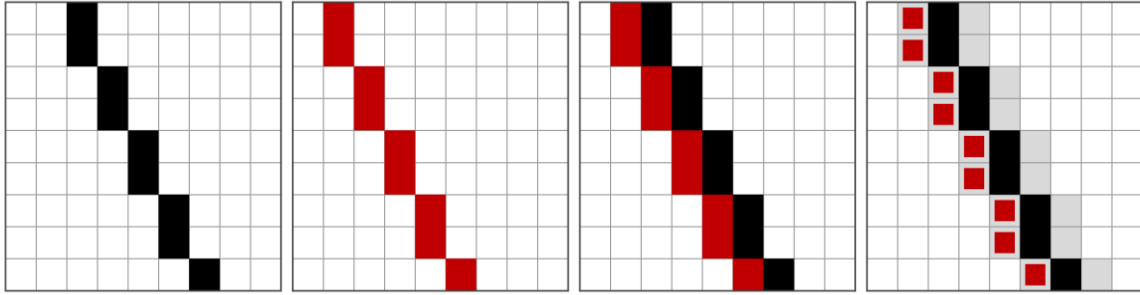


Fig. 6. Illustration of the tolerance τ used in the evaluation metric. Left to right: the ground-truth line (black), a prediction displaced by one pixel (red), the two superimposed, and the same pair with the $\tau = 1$ tolerance band (grey).

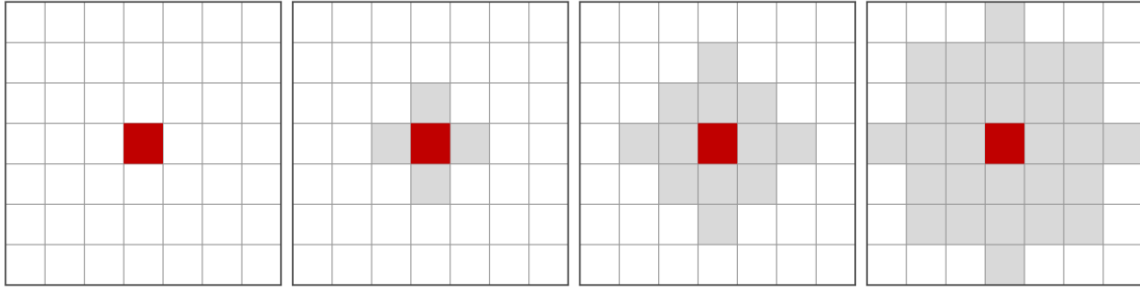


Fig. 7. Tolerance region around a single predicted pixel (red) for $\tau = 0, 1, 2$ and 3 (left to right). Grey pixels lie within a Euclidean distance τ of the predicted pixel. Diagonal neighbours (distance $\sqrt{2} \approx 1.41$ px) are therefore included only for $\tau \geq 2$.

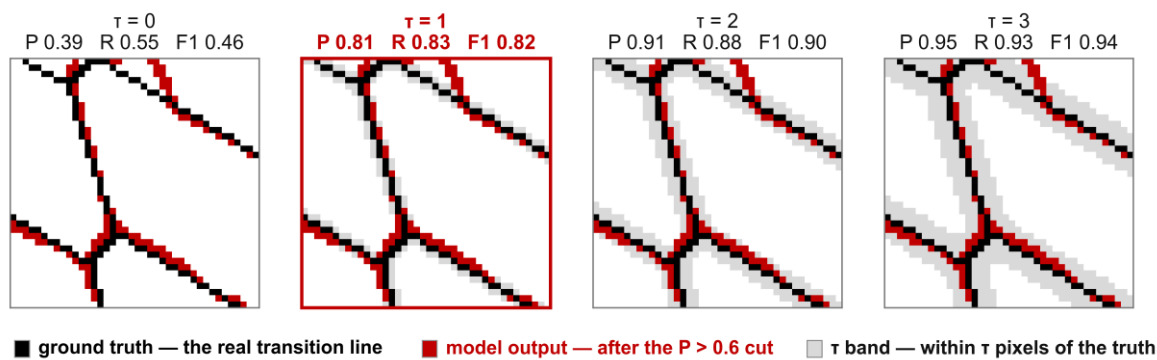


Fig. 8. One prediction scored at $\tau = 0-3$. Black: ground truth; red: output at $P > 0.6$; grey: pixels within τ of the ground truth. $\tau = 1$ is highlighted.

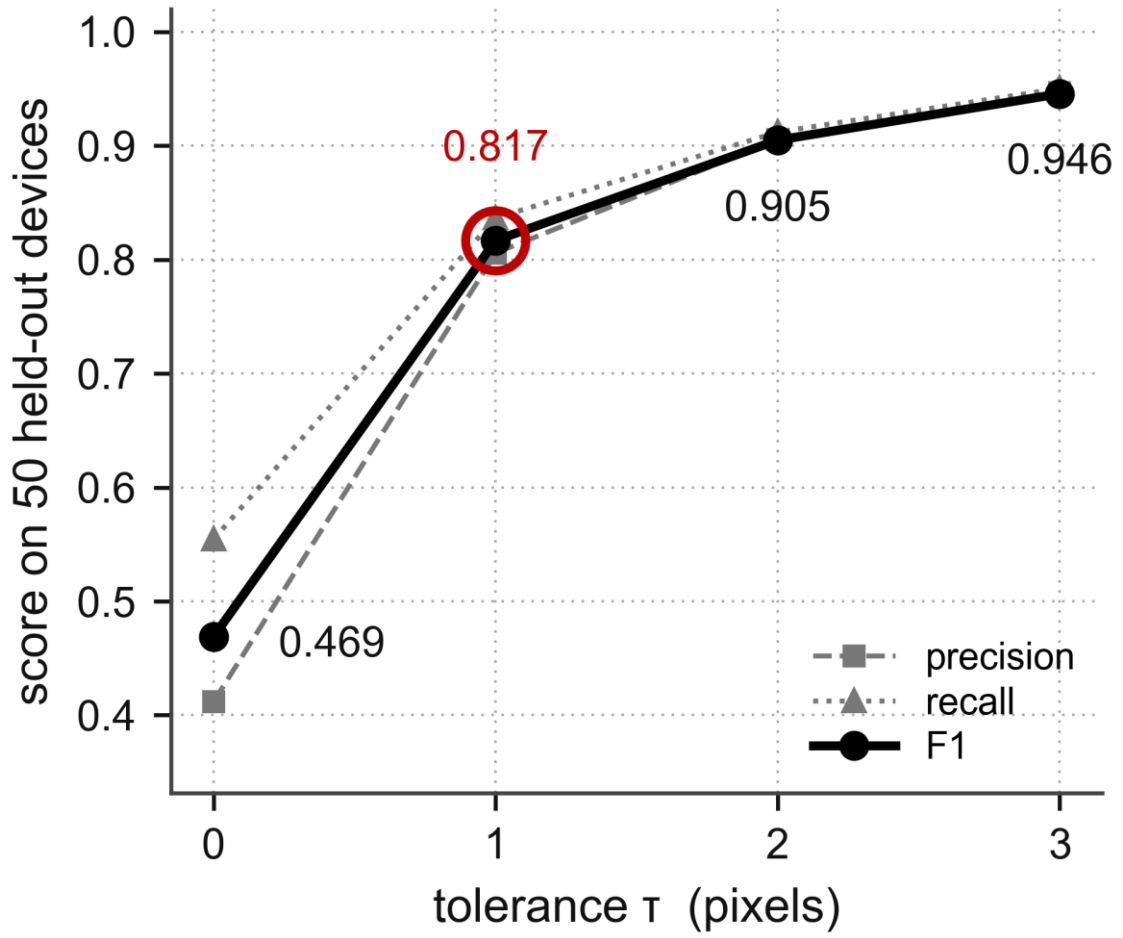


Fig. 9. Precision, recall and F1 score as functions of the tolerance τ for the 8×60 measurement budget, averaged over the 50 test devices. The largest gain occurs between $\tau = 0$ and $\tau = 1$. For larger τ the scores increase only slightly.